

Yes. I think this is a good candidate for **Flutter**, especially because you want the same application on **mobile + desktop**.

Let's define the **MRD (Minimum Requirements Document)** first, without getting into implementation yet.

Workout App — MRD v0.1

1. Exercises

The application contains a reusable exercise library.

Exercise

Each exercise should have:

- `id`
- `name`
- `description`
- `instructions`
- `photo`
- `category`
- `difficulty`
- `duration` — optional
- `repetitions` — optional
- `rest` — optional
- `targetMuscles` — optional
- `equipment` — default: none
- `active` — whether the exercise is currently available

Categories

Initially:

- Mobility / ניידות
- Warm-up / חימום
- Strength / כוח
- Cardio / אירובי
- Stretching / מתיחות
- Cool-down / שחרור

The exercise should support **multiple languages**, at least:

- Hebrew

- English

So instead of:

```
name: "Squat"
```

we should eventually have something like:

```
name:  
  en: "Squat"  
  he: "טריינינג"
```

2. Users

We need two levels of users.

User

```
User  
├─ id  
├─ name  
├─ email  
├─ photo  
├─ birthDate  
├─ gender  
├─ fitnessLevel  
├─ goals  
├─ language  
├─ createdAt  
└─ active
```

Fitness profile

This is important because the application should eventually be able to build an appropriate training program.

For example:

```
fitnessLevel:  
  beginner  
  intermediate  
  advanced
```

Goals could be:

```
general fitness
strength
mobility
cardio
weight management
```

We can make these extensible rather than hard-code them.

3. User → Training Programs

This is the most important part.

A **Training Program** is a plan assigned to a specific user.

For example:

"Weekly Program"

```
User: Yair

Monday
  Warm-up
  Squats
  Wall Push-ups
  Glute Bridge
  Cardio
  Cool-down

Wednesday
  Mobility
  Squats
  Lunges
  Cardio
  Cool-down

Friday
  Full Body
  ...
```

But I would make the data model slightly more general.

Program

```
Program
├─ id
├─ userId
├─ name
```

```
| description
| startDate
| endDate
| schedule
└ active
```

Program Day

```
ProgramDay
| id
| programId
| date / weekday
| title
└ exercises
```

Program Exercise

This is important:

```
ProgramExercise
| exerciseId
| order
| duration
| repetitions
| sets
| rest
└ notes
```

This means the same exercise can appear differently in different programs.

For example:

Squat

Program A:

```
10 repetitions × 2 sets
```

Program B:

```
15 repetitions × 3 sets
```

The exercise itself doesn't change.

4. User → Actual Achievement

I would call this **Workout Session / Workout Record** rather than simply "achievement".

The distinction is important:

Program = what the user was supposed to do

Achievement = what the user actually did

Example:

Planned

Friday

Squat

2 × 12

Wall Push-up

2 × 10

Glute Bridge

2 × 12

Actual

Friday 28/08/2026

Squat

Set 1: 12 ✓

Set 2: 10 ✓

Wall Push-up

Set 1: 10 ✓

Set 2: 10 ✓

Glute Bridge

Set 1: 12 ✓

Set 2: 12 ✓

So I suggest:

WorkoutSession

WorkoutSession

└ id

└ userId

└ programDayId

└ startedAt

```
|— completedAt
|— status
|— duration
└— exercises
```

And:

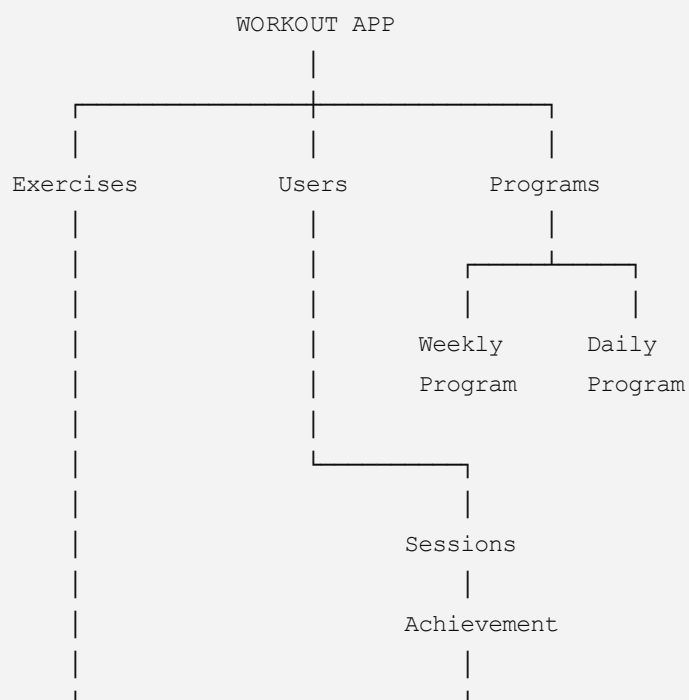
WorkoutExerciseResult

```
WorkoutExerciseResult
|— exerciseId
|— plannedSets
|— actualSets
|— plannedRepetitions
|— actualRepetitions
|— plannedDuration
|— actualDuration
|— completed
└— notes
```

This gives us a very useful historical record.

5. The application structure

I would therefore define the first version as:



6. Main screens

For the **MVP**, I would keep the UI relatively small.

User side

Home

|

└ Today's Workout

|

└ Start Workout

|

└ My Program

|

└ History

|

└ Profile

Exercise library

Exercises

|

└ All

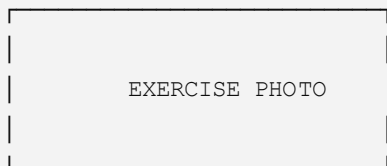
└ Mobility

└ Strength

└ Cardio

└ Stretching

Selecting an exercise:



Squat

ᵁᵛᵛᵁ

Difficulty: Beginner

How to perform

1. ...

2. ...

3. ...

Target muscles

...

[Add to Program]

7. Training screen

This is where I think the application can become much better than a simple PDF.

Example:

TODAY'S WORKOUT

20:00

EXERCISE PHOTO
SQUAT
12 reps
[START]

Exercise 1 / 8

- Warm-up
- Squat
- Push-up
- Lunges
- ...

When the user presses **START**, the application can actually guide them through the workout.

For a timed exercise:

SQUAT

00:23



12 repetitions

[DONE]

For example, the original 20-minute workout could become an **interactive workout**, rather than just instructions on a screen.

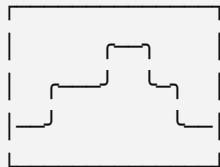
8. Achievement / History

This should eventually provide useful statistics.

MY PROGRESS

This week

Workouts	3
Total time	61 min
Exercises	18
Completion	92%



And:

WORKOUT HISTORY

28 Aug	Full Body	✓
26 Aug	Mobility	✓
24 Aug	Full Body	✓
21 Aug	Cardio	80%

9. One important addition

I would add a concept that isn't explicitly in your list:

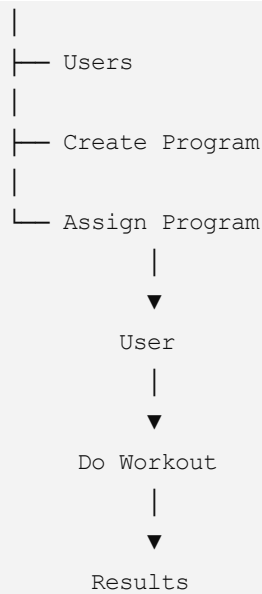
Trainer / Administrator

Because "**pre-user training program**" suggests that somebody may create programs for users.

So:

Administrator / Trainer

|
└ Exercise Library



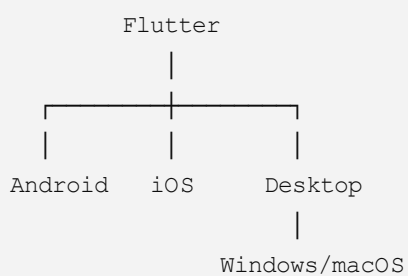
This opens the possibility of eventually having:

- one person managing several users
- predefined programs
- personalized programs
- exercise library management
- progress monitoring

10. Flutter architecture

For this project, **I would choose Flutter.**

The reason is that your requirement is unusually well suited to it:



And we can keep the business model completely independent of the UI.

I would structure the project roughly like:

```
lib/  
|  
├─ core/  
|   ├─ theme/  
|   └─ localization/
```

```

|   └─ utilities/
|
├─ models/
|   └─ user.dart
|   └─ exercise.dart
|   └─ program.dart
|   └─ program_day.dart
|   └─ workout_session.dart
|   └─ workout_result.dart
|
├─ features/
|   └─ exercises/
|   └─ users/
|   └─ programs/
|   └─ workouts/
|   └─ history/
|   └─ profile/
|
└─ main.dart

```

One architectural decision I'd make now

Do not make the application dependent on a backend yet.

We can design the domain model so that it works first with a local database:

```

Flutter
|
├─ UI
|
├─ Business logic
|
└─ Repository
    └─ Local DB

```

Later:

```

Flutter
|
└─ Repository
    |
    └─ Local DB
        |
        └─ API
            |
            └─ Backend

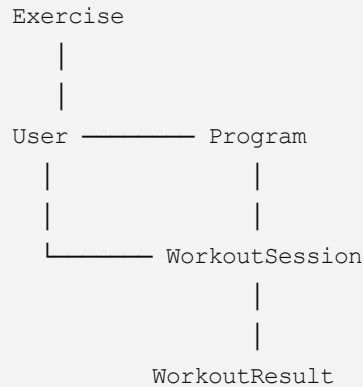
```

That gives us the freedom to decide later between **SQLite/Drift, Isar, Supabase, Firebase, NestJS/MongoDB**, etc.

MRD v0.1 — Core entities

So I would currently freeze the conceptual model as:

5 primary entities



with **Exercise** → **ProgramExercise** → **WorkoutResult** providing the connection between the exercise library, what was planned, and what was actually performed.

The next thing I'd define is the **exact data model + screens/workflows** before writing any Flutter code. That will prevent us from painting ourselves into a corner later.